25VL612 DIGITAL VLSI TESTING AND TESTABILITY LAB

EXPERIMENT-1

Name	:	Vedasri J
Roll No	:	CB.EN.P2VLD25035
Date	:	19/02/2026

Expt No: 1 **Equivalence Fault Collapsing** **Date:20-02-2026**

Evaluation Table			Faculty Signature
Code:	Output:	Record:	

1. AIM:
To implement equivalence fault collapsing for a combinational benchmark circuit and compute the collapse ratio.

2. SOFTWARE/TOOLS USED:
Google Colab (Python)

3. EXPERIMENTAL SETUP:
3.1. Benchmark Circuits
To evaluate the implemented equivalence fault collapsing algorithm, standard benchmark circuits from the ISCAS-85 suite were used. The circuits uploaded include c17, c432, c499, c880, and c1355. These benchmarks differ in size and structural complexity.

For demonstration in this report, detailed results are presented only for c17, a small 5-input, 2-output combinational circuit. The remaining circuits are uploaded in the Colab environment to allow user-selected execution and further testing.

3.2. Simulation Environment
The implementation was developed and executed using Python in Google Colab. The benchmark Verilog files were uploaded into the Colab runtime environment using the files.upload() utility, as shown in Figure 1. Upon upload, the files were stored in the working directory.

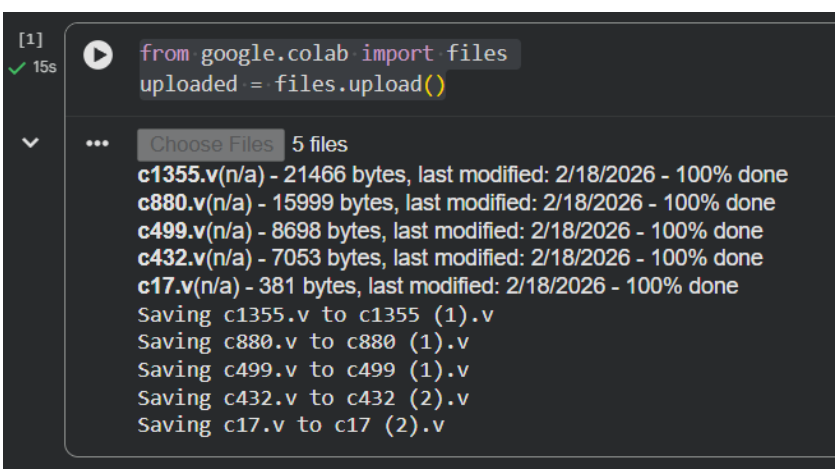

Figure 1: Uploaded ISCAS-85 benchmark Verilog files into Google Colab environment.

3.3. Fault Model
The classical single stuck-at fault model was adopted. For each signal line, two faults were considered: stuck-at-0 (sa0) and stuck-at-1 (sa1). In the presence of fanout structures, faults were modeled separately for the fanout stem and each individual branch. Consequently, the initial fault set consists of two faults per line, with additional faults for each fanout branch.

3.4. Fault Collapsing Technique

Structural equivalence fault collapsing was implemented based on gate-level equivalence properties:

- For AND/NAND gates: input sa0 faults are equivalent.
- For OR/NOR gates: input sa1 faults are equivalent.
- For NOT/BUF gates: both input faults collapse appropriately.
- XOR/XNOR gates were not collapsed structurally due to lack of simple equivalence relations.

The algorithm performs:

i. Levelization of the circuit
ii. Construction of the complete fault set
iii. Gate-wise equivalence collapsing
iv. Generation of:
    - Collapsed Fault Set
    - Eliminated Fault Set
    - Collapse Ratio

3.5. Evaluation Metric

The effectiveness of collapsing is quantified using the Collapse Ratio:

$$\text{Collapse Ratio} = \frac{|\text{ Collapsed Fault Set }|}{|\text{ Initial Fault Set }|}$$

Where:

- Initial Fault Set = Total stuck-at faults before collapsing
- Collapsed Fault Set = Remaining faults after equivalence reduction

4. CODE:

```
import copy

# ==============================================================
# READ VERILOG FILE
# ==============================================================

verilog_file = input("Enter Verilog file name: ")
file_handle = open(verilog_file, "r")

gate_data = []
fanin_list = []
fanout_list = []

for raw_line in file_handle:
    raw_line = raw_line.strip()

    if
raw_line.startswith(("and","or","nand","nor","xor","xnor","not","b
uf")):

        raw_line = raw_line.replace(";", "")
        lhs, rhs = raw_line.split("(")
```

```
        tokens = lhs.split()
        gate_kind = tokens[0]
        gate_id = tokens[1]

        rhs = rhs.replace(")", "")
        signal_list = [p.strip() for p in rhs.split(",")]

        out_signal = signal_list[0]
        in_signals = signal_list[1:]

        fanout_list.append(out_signal)

        for sig in in_signals:
            fanin_list.append(sig)

        gate_data.append([gate_kind, gate_id, out_signal,
in_signals])

file_handle.close()

# ==============================================================
# LEVELIZATION
# ==============================================================

level_map = {}

for sig in fanin_list:
    if sig not in fanout_list:
        level_map[sig] = 0

update_flag = True
while update_flag:
    update_flag = False

    for entry in gate_data:
        out_sig = entry[2]
        in_sigs = entry[3]

        if all(x in level_map for x in in_sigs):
            computed_level = max(level_map[x] for x in in_sigs) +
1
            if out_sig not in level_map:
                level_map[out_sig] = computed_level
                update_flag = True


# ==============================================================
# BUILD FAULT MODEL
# ==============================================================
```

```
fanout_counter = {}
for sig in fanin_list:
    fanout_counter[sig] = fanout_counter.get(sig, 0) + 1

fault_dict_initial = {}
fault_dict_current = {}

for sig, freq in fanout_counter.items():
    if freq == 1:
        fault_dict_initial[sig] = ["sa0", "sa1"]
    else:
        fault_dict_initial[sig] = {
            "stem": ["sa0", "sa1"],
            "branch": [["sa0", "sa1"] for _ in range(freq)]
        }

for sig in fanout_list:
    if sig not in fault_dict_initial:
        fault_dict_initial[sig] = ["sa0", "sa1"]

fault_dict_current = copy.deepcopy(fault_dict_initial)

# ============================================================
# HELPERS
# ============================================================

def compute_fault_count(fault_table):
    total_faults = 0
    for entry in fault_table.values():
        if isinstance(entry, list):
            total_faults += len(entry)
        else:
            total_faults += len(entry["stem"])
            for br in entry["branch"]:
                total_faults += len(br)
    return total_faults

def drop_input_fault(signal_name, stuck_fault):
    entry = fault_dict_current[signal_name]

    if isinstance(entry, list):
        if stuck_fault in entry:
            entry.remove(stuck_fault)
    else:
        for br in entry["branch"]:
            if stuck_fault in br:
                br.remove(stuck_fault)
                break
```

```
def extract_faults(fault_table):
    flat_list = []
    for sig, entry in fault_table.items():
        if isinstance(entry, list):
            for f in entry:
                flat_list.append((f"{sig}_line", f))
        else:
            for f in entry["stem"]:
                flat_list.append((f"{sig}_stem", f))
            for idx, br in enumerate(entry["branch"]):
                for f in br:
                    flat_list.append((f"{sig}_branch{idx+1}", f))
    return sorted(flat_list)

def display_fault_table(title, fault_table):
    print("\n" + "="*60)
    print(title)
    print("="*60)
    print("{:<20} {:<20}".format("Node_Location","Faults"))
    print("-"*60)

    for sig, entry in sorted(fault_table.items()):
        if isinstance(entry, list):
            print("{:<20} {:<20}".format(
                f"{sig}_line",
                ", ".join(entry) if entry else "-"
            ))
        else:
            print("{:<20} {:<20}".format(
                f"{sig}_stem",
                ", ".join(entry["stem"]) if entry["stem"] else "-"
            ))
            for idx, br in enumerate(entry["branch"]):
                print("{:<20} {:<20}".format(
                    f"{sig}_branch{idx+1}",
                    ", ".join(br) if br else "-"
                ))

    print("-"*60)
    print("Total Faults:", compute_fault_count(fault_table))

# ============================================================
# INITIAL FAULT TABLE
# ============================================================

display_fault_table("INITIAL FAULT TABLE", fault_dict_initial)

initial_faults = extract_faults(fault_dict_initial)
```

```
# ============================================================
# GATE-WISE COLLAPSING
# ============================================================

print("\n" + "="*60)
print("GATE-WISE EQUIVALENCE COLLAPSING")
print("="*60)

max_depth = max(level_map.values())

for depth in range(1, max_depth + 1):
    for entry in gate_data:

        g_type = entry[0]
        out_sig = entry[2]
        in_sigs = entry[3]

        if level_map[out_sig] != depth:
            continue

        print("\nGate: {:<6} Output: {:<6}".format(g_type.upper(),
out_sig))
        print("Faults Before:",
compute_fault_count(fault_dict_current))

        if g_type in ("and","nand"):
            for sig in in_sigs:
                drop_input_fault(sig, "sa0")

        elif g_type in ("or","nor"):
            for sig in in_sigs:
                drop_input_fault(sig, "sa1")

        elif g_type in ("not","buf"):
            for sig in in_sigs:
                drop_input_fault(sig, "sa0")
                drop_input_fault(sig, "sa1")

        print("Faults After :",
compute_fault_count(fault_dict_current))

# ============================================================
# COLLAPSED FAULT SET & ELIMINATED FAULTS
# ============================================================

final_faults = extract_faults(fault_dict_current)

collapsed_set = set(final_faults)
eliminated_set = sorted(set(initial_faults) - collapsed_set)
```

```
print("\n" + "="*60)
print("COLLAPSED FAULT SET")
print("="*60)
for loc, flt in sorted(collapsed_set):
    print(f"{loc:<20} {flt}")
print("Total Collapsed Faults:", len(collapsed_set))

print("\n" + "="*60)
print("ELIMINATED FAULTS")
print("="*60)
for loc, flt in eliminated_set:
    print(f"{loc:<20} {flt}")
print("Total Eliminated Faults:", len(eliminated_set))

# ============================================================
# SUMMARY
# ============================================================

initial_total = len(initial_faults)
final_total = len(final_faults)

print("\n" + "="*60)
print("COLLAPSING SUMMARY")
print("="*60)
print("Initial Faults     :", initial_total)
print("Collapsed Fault Set:", final_total)
print("Collapse Ratio     :", round(final_total / initial_total,
4))
print("="*60)
```

5. OUTPUT:

The below output corresponds to the ISCAS benchmark circuit c17.v

```
Enter Verilog file name: c17.v

============================================================
INITIAL FAULT TABLE
============================================================
Node_Location        Faults
------------------------------------------------------------
N1_line              sa0, sa1
N10_line             sa0, sa1
N11_stem             sa0, sa1
N11_branch1          sa0, sa1
N11_branch2          sa0, sa1
N16_stem             sa0, sa1
N16_branch1          sa0, sa1
N16_branch2          sa0, sa1
N19_line             sa0, sa1
N2_line              sa0, sa1
N22_line             sa0, sa1
N23_line             sa0, sa1
N3_stem              sa0, sa1
N3_branch1           sa0, sa1
N3_branch2           sa0, sa1
N6_line              sa0, sa1
N7_line              sa0, sa1
------------------------------------------------------------
Total Faults: 34
```

```
============================================================
GATE-WISE EQUIVALENCE COLLAPSING
============================================================

Gate: NAND   Output: N10
Faults Before: 34
Faults After : 32

Gate: NAND   Output: N11
Faults Before: 32
Faults After : 30

Gate: NAND   Output: N16
Faults Before: 30
Faults After : 28

Gate: NAND   Output: N19
Faults Before: 28
Faults After : 26

Gate: NAND   Output: N22
Faults Before: 26
Faults After : 24

Gate: NAND   Output: N23
Faults Before: 24
Faults After : 22
```

```
============================================================
COLLAPSED FAULT SET
============================================================
N10_line            sa1
N11_branch1         sa1
N11_branch2         sa1
N11_stem            sa0
N11_stem            sa1
N16_branch1         sa1
N16_branch2         sa1
N16_stem            sa0
N16_stem            sa1
N19_line            sa1
N1_line             sa1
N22_line            sa0
N22_line            sa1
N23_line            sa0
N23_line            sa1
N2_line             sa1
N3_branch1          sa1
N3_branch2          sa1
N3_stem             sa0
N3_stem             sa1
N6_line             sa1
N7_line             sa1
Total Collapsed Faults: 22
```

```
============================================================
ELIMINATED FAULTS
============================================================
N10_line            sa0
N11_branch1         sa0
N11_branch2         sa0
N16_branch1         sa0
N16_branch2         sa0
N19_line            sa0
N1_line             sa0
N2_line             sa0
N3_branch1          sa0
N3_branch2          sa0
N6_line             sa0
N7_line             sa0
Total Eliminated Faults: 12

============================================================
COLLAPSING SUMMARY
============================================================
Initial Faults     : 34
Collapsed Fault Set: 22
Collapse Ratio     : 0.6471
============================================================
```

6. INFERENCE:

The stuck-at fault model was successfully generated and equivalence fault collapsing was performed on the given circuits. The implementation was verified using the ISCAS benchmark circuit **c17.v**, and the collapsed fault count was obtained correctly as per theoretical expectations. This experiment demonstrates the correctness of the equivalence fault collapsing algorithm and its effectiveness in reducing redundant faults without affecting fault coverage.